OS LAB ASSIGNMENT-5

GROUP-18:

Vislavath Bindhu (230101112)
Tippana Likitha Reddy (230101105)
Bommagani Rohitha (230101027)
Annadevara Dedeepya Siri (230101011)

Drive link for deliverables:
https://drive.google.com/drive/folders/1NRhOGUtQ9gbrKcjw2ok8wGXqnOnCPkRQ?usp=drive_link

Task 5.1: Implement Doubly-Indirect Block Support in xv6 Objective:

The xv6 file system originally supports 12 direct block pointers and one single indirect block pointer per inode. This limits the maximum file size to ~268 KB. The goal of this task was to extend the file system to support larger files by adding a doublyindirect block pointer.

1. Modifications to fs.h:-

- Direct blocks reduced: From 12 → 11 (NDIRECT = 11).
- Single indirect block retained: Positioned at addrs[NDIRECT].
- Doubly-indirect block added: Positioned at addrs[NDIRECT + 1].

```
42 #define NDIRECT 11
43 #define NINDIRECT (BSIZE / sizeof(uint))
44 #define MAXFILE (NDIRECT + NINDIRECT + NINDIRECT * NINDIRECT)
45
46 struct dinode {
47   short type;
48   short major;
49   short minor;
50   short nlink;
51   uint size;
52   uint addrs[NDIRECT + 2]; // direct[0..NDIRECT-1], single indirect at addrs[NDIRECT],
   doubly at addrs[NDIRECT+1]
53 };
```

2. Modifications to fs.c – bmap():

The bmap() function is responsible for mapping a logical block number (within a file) to a physical disk block. If the corresponding physical block is not yet allocated, it allocates it on the fly. With the addition of a doubly-indirect block, bmap() now supports much larger files.

1. Direct Blocks The inode has a fixed number of direct block pointers (NDIRECT).

- If the requested block number bn is smaller than NDIRECT, it is stored directly in the inode.
- If the block is not allocated yet, balloc() allocates a free disk block.
- Example: If NDIRECT = 11, logical blocks 0–10 use direct pointers.

Advantage: Access is very fast because the block address is stored directly in the inode.

2. Single Indirect Block

- Blocks beyond the direct pointers are handled by a single indirect block.
- The inode field addrs[NDIRECT] points to an indirect block, which contains an array of 256 block addresses.
- Steps: o If the indirect block itself doesn’t exist, allocate it. o Read the indirect block from disk using bread(). o If the requested data block is not allocated, allocate it and update the indirect block.
    - o Release the buffer with brelse().
- Example: Logical block 11 onwards (up to 11 + 256) are addressed via this single level of indirection.

Advantage: Allows the file to grow beyond the direct block limit.
3. Doubly-Indirect Block

- For even larger files, blocks beyond the single indirect range use a doubly-indirect block.
- The inode field addrs[NDIRECT+1] points to a doubly-indirect block, which contains 256 pointers to indirect blocks.
- Each of these indirect blocks contains 256 pointers to actual data blocks.

Steps in detail:

1. Allocate or read the doubly-indirect block if it does not exist.
2. Compute indices:
    - dindirect_index → which indirect block to use.
    - indirect_index → which data block within that indirect block. o Allocate or read the indirect block pointed to by dindirect_index.
    - Allocate the data block if it does not exist.
    - Return the physical disk address.

4. Out-of-Range Handling

- If the logical block number exceeds the maximum supported by direct, single indirect, and doubly-indirect pointers, the kernel triggers a panic.
- This prevents invalid memory access and ensures file system integrity.

```
433 static uint bmap(struct inode *ip, uint bn) {
434   uint addr, *a;
435   struct buf *bp, *bp2;
436   if(bn < NDIRECT){
437     // Same as before: direct blocks
438     if((addr = ip->addrs[bn]) == 0)
439       ip->addrs[bn] = addr = balloc(ip->dev);
440     return addr;
441   }
442   bn -= NDIRECT;
443   // Single indirect
444   if(bn < NINDIRECT){
445     if((addr = ip->addrs[NDIRECT]) == 0)
446       ip->addrs[NDIRECT] = addr = balloc(ip->dev);
447     bp = bread(ip->dev, addr);
448     a = (uint*)bp->data;
449     if((addr = a[bn]) == 0){
450       a[bn] = addr = balloc(ip->dev);
451       log_write(bp);
452     }
453     brelse(bp);
454     return addr;
455   }
456   bn -= NINDIRECT;
457   // Doubly-indirect
458   if(bn < NDINDIRECT){
459     // 1. Allocate doubly-indirect block
460     if((addr = ip->addrs[NDIRECT+1]) == 0)
461       ip->addrs[NDIRECT+1] = addr = balloc(ip->dev);
462     bp = bread(ip->dev, addr);
463     a = (uint*)bp->data;
464     uint dindirect_index = bn / NINDIRECT;
465     uint indirect_index  = bn % NINDIRECT;
467     // 2. Allocate indirect block
468     if((addr = a[dindirect_index]) == 0){
469       a[dindirect_index] = addr = balloc(ip->dev);
470       log_write(bp);
471     }
472     brelse(bp);
473     bp2 = bread(ip->dev, addr);
474     a = (uint*)bp2->data;
475
476     // 3. Allocate data block
477     if((addr = a[indirect_index]) == 0){
478       a[indirect_index] = addr = balloc(ip->dev);
479       log_write(bp2);
480     }
481     brelse(bp2);
482     return addr;
483   }
484   panic("bmap: out of range");
485   return -1;
486 }
```

3. Modifications to fs.c – itrunc()

   The itrunc() function truncates a file by freeing all the disk blocks associated with an inode. This includes direct, single indirect, and doubly-indirect blocks, and resets the file size to zero.

1. Direct blocks: Iterates through all direct block pointers in the inode and frees each allocated block.
2. Single indirect block: Reads the indirect block from disk, frees each data block it references, then frees the indirect block itself.
3. Doubly-indirect block: Reads the doubly-indirect block, and for each indirect block it points to: a. Reads the indirect block.
    b. Frees all its data blocks.
    c. Frees the indirect block itself.
    d. Finally, frees the doubly-indirect block.
4. Finalization: Resets the inode's size to zero and updates the inode on disk with iupdate().

This recursive and orderly deallocation ensures that all allocated blocks are released, preventing memory leaks and maintaining file system integrity, even for very large files.

Together, bmap() and itrunc() provide a robust mechanism for dynamic block allocation and deallocation, enabling the file system to efficiently handle large files while keeping disk space management consistent and safe.

```
528 void itrunc(struct inode *ip) {
529   int i, j, k;
530   struct buf *bp, *bp2;
531   uint *a, *a2;
532   // Free direct blocks
533   for(i = 0; i < NDIRECT; i++) {
534     if(ip->addrs[i]) {
535       bfree(ip->dev, ip->addrs[i]);
536       ip->addrs[i] = 0;
537     }
538   }
539
540   // Free single indirect
541   if(ip->addrs[NDIRECT]) {
542     bp = bread(ip->dev, ip->addrs[NDIRECT]);
543     a = (uint*)bp->data;
544     for(j = 0; j < NINDIRECT; j++)
545       if(a[j])
546         bfree(ip->dev, a[j]);
547     brelse(bp);
548     bfree(ip->dev, ip->addrs[NDIRECT]);
549     ip->addrs[NDIRECT] = 0;
550   }
551
552   // Free doubly-indirect blocks
553   if(ip->addrs[NDIRECT+1]) {
554     bp = bread(ip->dev, ip->addrs[NDIRECT+1]);
555     a = (uint*)bp->data;
556     for(j = 0; j < NINDIRECT; j++) {
557       if(a[j]) {
558         bp2 = bread(ip->dev, a[j]);
559         a2 = (uint*)bp2->data;
560         for(k = 0; k < NINDIRECT; k++)
561           if(a2[k])
562             bfree(ip->dev, a2[k]);
563         brelse(bp2);
564         bfree(ip->dev, a[j]);
565       }
566     }
567
568     brelse(bp);
569     bfree(ip->dev, ip->addrs[NDIRECT+1]);
570     ip->addrs[NDIRECT+1] = 0;
571   }
572
573   ip->size = 0;
574   iupdate(ip);
575 }
```

4. Bigfile.c:

Purpose:

- Test the doubly-indirect block support in the modified xv6 file system.
- Create a file larger than the original xv6 limit (~268 KB) to verify allocation across direct, single indirect, and doubly-indirect blocks.

File Creation:

- Opens or creates a file named "bigfile" with read/write access. Write Phase:
- Writes 600 KB of data in 1 KB blocks (FILESIZE = 1024 * 600).
- Fills each block with a predictable pattern: (i + j) & 0xFF.
- Allocates new blocks automatically using the modified bmap() function.
- Prints progress every 64 blocks (written X blocks).

Read and Verify Phase:

- Reopens the file in read-only mode.
- Reads each 1 KB block sequentially.
- Verifies each byte matches the expected pattern (i + j) & 0xFF.
- Prints progress every 64 blocks (read verified X blocks). Error Handling:
- If a write or read fails, or if data verification fails, the program prints an error message and exits.

5. OUTPUT :

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ bigfile
written 0 blocks
written 64 blocks
written 128 blocks
written 192 blocks
written 256 blocks
written 320 blocks
written 384 blocks
written 448 blocks
written 512 blocks
written 576 blocks
write done: 600 blocks
read verified 0 blocks
read verified 64 blocks
read verified 128 blocks
read verified 192 blocks
read verified 256 blocks
read verified 320 blocks
read verified 384 blocks
read verified 448 blocks
read verified 512 blocks
read verified 576 blocks
read verify done OK
$ QEMU: Terminated
```

6. Doubly-Indirect Addressing Implementation:

Doubly-indirect pointer → points to a doubly-indirect block, which contains an array of pointers to indirect blocks, each of which contains data block addresse Doubly-indirect addressing adds a second level of indirection on top of single indirect blocks.

- Allows very large files to be stored efficiently while maintaining a fixed inode size (128 bytes).
- Combined with bmap() for allocation and itrunc() for deallocation, it ensures correct and safe file system operations for large files.

7. Theoretical Maximum File Size:

- Direct blocks: 11 × 1024 B = 11,264 B

- Single indirect: 256 × 1024 B = 262,144 B
- Doubly indirect: 256 × 256 × 1024 B = 67,108,864 B
- Total MAXFILE: NDIRECT + NINDIRECT + NINDIRECT * NINDIRECT
  11264+ 262144 + 67108864 = 67,382,272

Task 5.2: Implementation of Symbolic (Soft) Links in xv6

Symlink Mechanism and Recursive Resolution

- A symbolic link (symlink) is a special file type that acts as a reference to another file or directory. Instead of containing actual data, it stores the path of the target file. When a process opens a symlink, xv6 internally reads this stored path and redirects the operation to the corresponding target file. This allows one file to serve as a convenient shortcut to another.
- In xv6, the symlink system call creates a new inode of type T_SYMLINK and saves the target's pathname inside it. During file access, if the opened inode is of type T_SYMLINK, the kernel automatically follows the stored path unless the O_NOFOLLOW flag is used.
- To correctly handle recursive or nested links, xv6 performs recursive resolution inside sys_open. When a link points to another link, the kernel repeatedly follows the target paths until it reaches a regular file. This enables smooth resolution for chains like link1 → link2 → link3 → file.txt.
- However, to prevent infinite loops due to circular links (for example, a → b and b → a), a maximum resolution depth of 10 was implemented. If this limit is exceeded, xv6 stops the process and returns an error, ensuring kernel safety and avoiding stack overflows.

1. Objective

The goal of this task was to implement symbolic links (soft links) in the xv6 file system. Symbolic links are special files that store a reference (path) to another file or directory. Programs can access the target transparently through the link.

Requirements implemented:

- Create symbolic links via a system call symlink(target, path).
- Recursively resolve symbolic links in open().
- Prevent infinite loops with a recursion depth limit.
- Support O_NOFOLLOW to open the link itself.
- Ensure unlink() removes only the symlink, not the target.

2. Kernel Changes

2.1 fcntl.h

```
#define O_NOFOLLOW 0x800
```

Explanation:

- Added O_NOFOLLOW to allow opening the symlink itself instead of its target.

2.2 stat.h

```
#define T_SYMLINK 4
```

Explanation:

- Defines a new inode type T_SYMLINK.
- Enables stat() and other utilities to recognize symbolic links.

2.3 fs.h

2.3.1 Updated inode structure

```c
// On-disk inode structure
struct dinode {
    ...
    uint addrs[NDIRECT+2]; // 10 direct + 1 single indirect + 1 double indirect
    ...
    short type;           // File type
    short major;          // Major device number (T_DEVICE only)
    short minor;          // Minor device number (T_DEVICE only)
    short nlink;          // Number of links to inode in file system
    uint size;            // Size of file (bytes)
    //uint addrs[NDIRECT+1];   // old
};
```

Explanation:

- addrs size increased to store small data (like a symlink target path).
- Enables writei() in sys_symlink() to store the target path in the inode.
- Supports the new T_SYMLINK type.

2.4 sysfile.c

2.4.1 sys_symlink()

```c
uint64 sys_symlink(void){
    char target[MAXPATH], path[MAXPATH];
    struct inode *ip;
    int len;
    if(argstr(0, target, MAXPATH) < 0 || argstr(1, path, MAXPATH) < 0){
        printf("sys_symlink: argstr failed\n");
        return -1;
    }
    for(len = 0; target[len] && len < MAXPATH; len++);
    begin_op();
    ip = create(path, T_SYMLINK, 0, 0);
    if(ip == 0){
        printf("sys_symlink: create failed\n");
        end_op();
        return -1;
    }
    int written = writei(ip, 0, (uint64)target, 0, len);
    printf("sys_symlink: writei returned %d (expected %d)\n", written, len);
    if(written != len){
        iunlockput(ip);
        end_op();
        return -1;
    }
    iunlockput(ip);
    end_op();
    printf("sys_symlink: success\n");
    return 0;
}
```

This function implements the symlink(target, path) system call. It:

- Fetches the target and path strings from user space.
- Creates a new inode at path with type T_SYMLINK.
- Stores the target path inside this inode as data.
- Returns 0 on success or -1 on failure.

Thus, the symbolic link is simply a file that contains the path to another file. The actual redirection is handled later during open().

2.4.2 sys_open() (symbolic link resolution)

```
if(ip->type == T_SYMLINK && !(omode & O_NOFOLLOW)){
    int depth = 0;

    while(ip->type == T_SYMLINK && depth < 10){
        char target[MAXPATH];
        int len;

        len = readi(ip, 0, (uint64)target, 0, MAXPATH - 1);
        if(len <= 0){
            printf("sys_open: readi failed, len=%d\n", len);
            iunlockput(ip);
            end_op();
            return -1;
        }

        target[len] = '\0';
        iunlockput(ip);

        if((ip = namei(target)) == 0){
            printf("sys_open: namei failed for target %s\n", target);
            end_op();
            return -1;
        }

        ilock(ip);
        depth++;
    }

    if(depth >= 10 && ip->type == T_SYMLINK){
        printf("sys_open: depth limit reached\n");
        iunlockput(ip);
        end_op();
        return -1;
    }
```

When opening a file, if it is a symbolic link (T_SYMLINK) and O_NOFOLLOW is not set, the system:

- Recursively reads the link's target path and resolves it.
- Limits recursion to 10 levels to avoid infinite loops.
- Returns an error if the target does not exist or the depth limit is exceeded.

If O_NOFOLLOW is set, the symlink itself is opened without resolution.

2.5 syscall.h

```
#define SYS_symlink 22      // Assign a unique number for symlink syscall
```

Explanation:

- Defines a unique syscall number for symlink.

2.5.2 syscall.c

```
extern uint64 sys_symlink(void);
```

- Declares the function prototype so the kernel knows about it.

```
[SYS_symlink]   sys_symlink,
```

- Registers the symlink system call in the system call table.
- Maps the syscall number (SYS_symlink) to the actual function (sys_symlink) so it can be invoked from user space.

3.USER Changes

3.1 usys.pl

```
entry("symlink");
```

Explanation:

- Adds the symlink syscall to user programs and shell.

3.2 user.h

```
int symlink(const char *target, const char *path);
```

Explanation:

- Provides the user-level interface to create symbolic links.

3.3 Test Program: symlinktest.c

The test program verifies all major symbolic link functionality. Summary of tests:

Test	Description	Expected Outcome
1	Create target and symlink	Symlink opens target correctly
2	O_NOFOLLOW	Opens symlink itself, not target
3	Circular symlinks	Detects loop and prevents infinite recursion
4	Multi-level symlink chain	Resolves all levels correctly
5	Unlink symlink	Removes link only; target remains

Short explanation:

- The test program creates symlinks, attempts to open them, follows multi-level chains, and tests circular detection.
- It also ensures unlinking affects only the symlink.
- All outputs confirm correct behavior of symbolic link operations.

4.MAKEFILE

```
UPROGS=\
    ...
    _symlinktest
```

Explanation:

- Ensures the test program symlinktest.c is compiled and included in the xv6 build.

5.Output:

- Symbolic links are fully functional in xv6.
- Recursive resolution works with a depth limit to prevent infinite loops.
- O_NOFOLLOW correctly opens the link itself.
- Circular links are detected.
- Unlinking affects only the symbolic link, leaving the target file intact.
- The system call symlink() is successfully integrated and validated through comprehensive testing.

```
hart 2 starting
hart 1 starting
sys_open: namei failed for console
sys_open: success, fd=0
init: starting sh
sys_open: success, fd=3
$ symlinktest
Starting symbolic link tests...

[Test 1] Creating target file and symbolic link...
sys_open: success, fd=3
sys_symlink: writei returned 11 (expected 11)
sys_symlink: success
sys_open: success, fd=3
[Test 1] Symbolic link works as expected.

[Test 2] Testing O_NOFOLLOW flag (should open symlink itself)..
sys_open: success, fd=3
[Test 2] O_NOFOLLOW works correctly.

[Test 3] Creating circular symlinks...
sys_symlink: writei returned 5 (expected 5)
sys_symlink: success
sys_symlink: writei returned 5 (expected 5)
sys_symlink: success
sys_open: depth limit reached
[Test 3] Circular links detected successfully.

[Test 4] Testing chain of symbolic links...
sys_symlink: writei returned 11 (expected 11)
sys_symlink: success
sys_symlink: writei returned 6 (expected 6)
sys_symlink: success
sys_symlink: writei returned 6 (expected 6)
sys_symlink: success
sys_open: success, fd=3
[Test 4] Chain of links resolved successfully.

[Test 5] Verifying unlinking symbolic links...
sys_symlink: writei returned 11 (expected 11)
sys_symlink: success
sys_open: namei failed for unlink_test
[Test 5] Symbolic link removed without affecting the target.

Symbolic link tests completed successfully.
$
```